KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Hotel Room Reservation and Guest Management System

Submitted by

KAVIN NILAVAN S

25BSC126

Under the Guidance of

Mr.ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "Hotel Room Reservation and Guest Management System" submitted in partial fulfillment of the requirements for the course JAVA PROGRAMMING is a record of my original work carried out under the guidance of Mr.ARAVIND, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

KAVIN NILAVAN S

CERTIFICATE

This is to certify that the project entitled “Hotel Room Reservation and Guest Management System” is a bona fide work carried out by KAVIN NILAVAN S, 25BSC126, BSC COMPUTER SCIENCE, KG College of Arts & Science, in partial fulfillment of the requirements for the course JAVA PROGRAMMING.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:10/04/2026

Submitted for the Viva-Voce Examination held on 10/04/2026

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
Mr.Aravind	Mr.Boopalan Sundaramoorthy

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**7**
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	**9**
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	**11**
3.1	Modules	
4	**Methodology**	**13**
4.1	Problem Analysis	

4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	**16**
6	**Input and Output Screens**	**19**
7	**Discussion and Conclusion**	**21**
7.1	Discussion	
7.2	Conclusion	
8	**References**	**22**

ABSTRACT

The Hotel Room Reservation and Guest Management System is a Java-based application designed to simplify hotel room booking and guest record management. Manual reservation methods can cause double bookings, incorrect room availability, and difficulty maintaining guest details. This project provides a simple menu-driven solution for managing these activities.

The system allows staff to view rooms, reserve an available room, record guest details, search reservations, view active bookings, calculate room charges, and check guests out. It is developed using Core Java concepts such as classes, objects, constructors, methods, ArrayList, loops, conditional statements, and input validation. The application runs in a console and does not require external libraries.

The main objective is to make room reservation and guest management easier, reduce manual errors, and calculate room charges accurately. The program maintains room availability while it is running; records are held in memory and are not saved after the application closes.

1.INTRODUCTION

Hotels need an organized way to manage room availability, guest information, reservations, and check-out. When these tasks are handled manually, staff may accidentally assign the same room to more than one guest or miscalculate the total room charge. Searching for an existing reservation can also take time.

The Hotel Room Reservation and Guest Management System is a Java-based application that automates basic hotel operations. It allows the user to display room details, reserve available rooms, enter guest information, search for a reservation, view active reservations, and complete check-out.

The project uses Core Java programming concepts, including classes, objects, methods, ArrayList, loops, switch statements, and exception handling for numeric input. It is a console-based application intended for learning and demonstration. Its data is stored in memory during execution.

1.1 Objective

The main objective of the Hotel Room Reservation and Guest Management System is to develop a simple Java application that manages room reservations and guest information accurately.

The specific objectives of this project are:

- To develop a menu-driven hotel management application using Java.
- To display room numbers, room types, prices, and availability.
- To reserve available rooms and generate reservation IDs.
- To store guest name, phone number, and address with each reservation.
- To search and display reservation details.
- To calculate the room bill based on price per night and number of nights.
- To release rooms after successful guest check-out and validate user input.

1.2 Overview

The Hotel Room Reservation and Guest Management System is a menu-driven console application written in Java. It provides a simple way for hotel staff to manage room bookings and guest records.

The user can view all rooms, reserve an available room, view active reservations, search by reservation ID, and check out a guest. A reservation records the guest details, assigned room, number of nights, and total room charge.

Room availability is updated after a successful reservation and restored after check-out. The system uses ArrayList collections to manage rooms and active reservations while the program is running. It does not require a database or external Java libraries.

This project demonstrates how object-oriented programming can be applied to a practical hotel management task.

The Hotel Room Reservation and Guest Management System includes the following main features:

1.3 Features

- **View Rooms – Displays room number, room type, nightly price, and availability.**
- **Room Reservation – Books an available room and records guest details.**
- **Guest Management – Stores guest name, phone number, and address.**
- **Reservation ID – Generates a unique sequential ID for each booking during the session.**
- **Reservation Search – Finds active reservation details using the reservation ID.**
- **Bill Calculation – Calculates charges using nightly price multiplied by the number of nights.**
- **Guest Check-out – Completes check-out and marks the room as available again.**
- **Input Validation – Handles invalid numeric input and rejects empty required fields.**

2.PROGRAM ANALYSIS

The Hotel Room Reservation and Guest Management System organizes room and reservation information in one menu-driven Java program. It checks room availability before booking, stores guest details, calculates room charges, and updates the room status during check-out.

The program is divided into classes for rooms, guests, reservations, and the main application. This structure keeps the code organized and makes each class responsible for a specific part of the system.

2.1 Problem Definition

Hotels require a reliable method to keep track of rooms and guest reservations. Manual records can be difficult to search and update, and may lead to duplicate room assignments, incomplete guest details, and billing mistakes. A simple computerized system can help staff view room status, register guests, retrieve bookings, and calculate the room charge consistently.

2.2 Project Overview

The Hotel Room Reservation and Guest Management System is developed using Java to support basic hotel operations through a console menu. It includes six sample rooms with different room types and prices.

When a user reserves a room, the system checks that the room exists and is available. It then collects the guest name, phone number, address, and number of nights. A reservation ID is generated and the room is marked occupied.

The program can list active reservations, search by reservation ID, and check out a guest. During check-out, the bill is displayed, the reservation is removed from the active list, and the room is made available again.

The project is suitable for demonstrating Core Java and basic object-oriented design. Because it uses in-memory collections, all reservation data is cleared when the program terminates.

2.3 Scope and Limitations

Scope

The system can be used as a basic demonstration tool for hotel room booking and guest record management. It supports room listing, reservation creation, guest detail entry, reservation search, bill calculation, and check-out.

The current version manages a fixed set of sample rooms and stores records only while the program is running. It does not include date-based booking, payments, user login, a database, or permanent storage.

Possible future enhancements include:

- User login and role-based access.
- Permanent storage using files or a database.
- Check-in and check-out dates with availability checks.

- Payment processing and invoice generation.
- Reports for occupancy, revenue, and guest history.

3.SYSTEM DESIGN

The Hotel Room Reservation and Guest Management System is designed as a menu-driven Java application. It contains a user interface for entering menu choices, application logic for handling reservations, and in-memory data structures for storing room and reservation information.
The system follows an object-oriented design with separate Room, Guest, and Reservation classes. The main application provides modules for displaying rooms, creating reservations, viewing and searching reservations, calculating bills, and checking out guests.

3.1 Modules

1. User Interface Module

Handles interaction between the staff member and the application. It displays menu options and collects the selected operation.

- View room details
- Reserve a room
- Search and view reservations; perform guest check-out.

2. Wallet Module

Represents each hotel room and stores the room number, type, nightly price, and availability status.

- Stores room number, room type, price, and status.
- Updates room availability after booking and check-out.

3. Transaction Management Module

Stores guest details such as name, phone number, and address.

- Collects guest name.
- Collects phone number and address.
- Associates guest details with a reservation.

Creates and manages booking information for each guest.

Stores reservation ID, guest, room, and number of nights.

- Displays complete reservation details.
- Calculates the total room charge.

Maintains the collection of room and reservation objects during execution.

Stores the predefined room list.

- Tracks active reservations.
- Uses ArrayList; information is not retained after program exit.

The in-memory data storage can be replaced with file or database storage in a future version.

Room Availability Module

Checks whether the selected room can be booked and updates its status after reservation or check-out.

- Displays available and occupied rooms.
- Prevents booking an occupied room.
- Releases the room after check-out.

Keeps room status consistent with active reservations.

4.METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For this project, a structured development approach was followed to identify requirements, design the system, implement the Java program, test the main operations, and prepare the report.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

` 4.1 Problem Analysis

The requirements were identified from common hotel booking tasks. The system needs to show room availability, register guest details, create reservation IDs, search active reservations, calculate charges, and release rooms after check-out.

The main problems identified in manual financial tracking are:

- Manual booking may result in duplicate room assignments.
- Guest information may be incomplete or difficult to retrieve.
- Room charges may be calculated incorrectly.
- Room availability may not be updated promptly.

The system was designed to address these issues using room status checks, reservation records, and automatic bill calculation.

4.2 System Design

In this stage, the overall structure of the system is designed before coding begins.

The application is divided into the following modules:

- Room Display Module
- Room Reservation and Guest Details Module
- Reservation Search and Listing Module
- Bill Calculation Module
- Guest Check-out and Room Release Module

A menu-driven structure allows the user to choose an operation and return to the menu until exit is selected.

4.3 Program Development

The application is implemented in Java using separate classes for Room, Guest, and Reservation, along with a main HotelReservationSystem class.

The following steps were followed during development:

- Creating classes and constructors for rooms, guests, and reservations.
- Implementing a menu using a switch statement and loop.
- Checking room existence and availability before reservation.
- Validating required text and numeric input.
- Displaying reservation details and calculating the bill.

ArrayList, encapsulation, methods, loops, conditionals, and exception handling are used in the implementation.

4.4 Testing and Validation

Testing verifies that each menu operation produces the expected result and that room status is updated correctly.

The following test cases can be used to validate the program.

Example Test Cases:

Test Case	Expected Result
View rooms	All room numbers, prices, and statuses are displayed
Reserve available room	Reservation ID is generated and room becomes occupied
Reserve occupied room	System displays an occupied-room warning
Guest check-out	Reservation is removed from active list and room becomes available

The test cases cover normal operations, unavailable rooms, invalid input, reservation lookup, and check-out.

4.5 Result and Documentation

After implementation, the program displays room information, reservation details, guest information, and the calculated room charge.

The system displays:

- Room status before and after reservation.
- Reservation ID, guest details, room details, and total bill.

The program can be demonstrated by compiling and running the Java file and capturing the console output for the report.

This report documents the objective, system design, methodology, algorithm, source code, sample input/output, and conclusion.

4.6 Algorithm

Algorithm for Hotel Room Reservation and Guest Management System:

1. Start the program and initialize the sample hotel rooms.
2. Display the menu options.
3. Read and validate the user's menu choice.
4. If View Rooms is selected, display all rooms and their availability.

5. If Reserve Room is selected, check availability, collect guest details and nights, create a reservation, and mark the room occupied.
6. If View or Search Reservation is selected, display the matching active reservation details.
7. If Check-out is selected, display the bill, confirm check-out, remove the active reservation, and mark the room available. Repeat until Exit is selected.
8. End the program.

4.7 Flowchart

Start

↓

Initialize Rooms and Display Menu

↓

Read Menu Choice

↓

View Rooms / Reserve / Search / Check-out

↓

Process Request and Display Result

↓

Exit Selected?

↓

End

5.SOURCE CODE

```
import java.util.ArrayList;
import java.util.Scanner;


class Room {
    private final int roomNumber;
    private final String roomType;
    private final double pricePerNight;
    private boolean available = true;


    Room(int roomNumber, String roomType, double pricePerNight) {
        this.roomNumber = roomNumber;
        this.roomType = roomType;
        this.pricePerNight = pricePerNight;
    }


    int getRoomNumber() { return roomNumber; }
    String getRoomType() { return roomType; }
    double getPricePerNight() { return pricePerNight; }
    boolean isAvailable() { return available; }
    void setAvailable(boolean value) { available = value; }


    void displayRoom() {
        System.out.printf("%-12d %-12s Rs.%-10.2f %-12s%n",
                roomNumber, roomType, pricePerNight,
                available ? "Available" : "Occupied");
    }
}
```

```
class Guest {
    private final String name;
    private final String phone;
    private final String address;

    Guest(String name, String phone, String address) {
        this.name = name;
        this.phone = phone;
        this.address = address;
    }

    String getName() { return name; }
    String getPhone() { return phone; }
    String getAddress() { return address; }
}

class Reservation {
    private final int id;
    private final Guest guest;
    private final Room room;
    private final int nights;

    Reservation(int id, Guest guest, Room room, int nights) {
        this.id = id;
        this.guest = guest;
        this.room = room;
        this.nights = nights;
    }

    int getId() { return id; }
    Guest getGuest() { return guest; }
    Room getRoom() { return room; }
    double calculateBill() { return room.getPricePerNight() * nights; }
```

```
    void display() {
        System.out.println("\n--------- RESERVATION DETAILS ---------");
        System.out.println("Reservation ID : " + id);
        System.out.println("Guest Name     : " + guest.getName());
        System.out.println("Phone Number   : " + guest.getPhone());
        System.out.println("Address        : " + guest.getAddress());
        System.out.println("Room Number    : " + room.getRoomNumber());
        System.out.println("Room Type      : " + room.getRoomType());
        System.out.println("Number of Nights: " + nights);
        System.out.printf("Total Bill     : Rs.%.2f%n", calculateBill());
        System.out.println("---------------------------------------");
    }
}

public class HotelReservationSystem {
    private static final Scanner sc = new Scanner(System.in);
    private static final ArrayList<Room> rooms = new ArrayList<>();
    private static final ArrayList<Reservation> reservations =
            new ArrayList<>();
    private static int nextId = 1001;

    public static void main(String[] args) {
        initializeRooms();
        System.out.println("Welcome to the Hotel Reservation System!");

        while (true) {
            showMenu();
            int choice = readInt("Enter your choice: ");

            switch (choice) {
                case 1: displayRooms(); break;
                case 2: reserveRoom(); break;
                case 3: viewReservations(); break;
                case 4: searchReservation(); break;
```

```
            case 5: checkOut(); break;
            case 6:
                System.out.println("Thank you for visiting our hotel!");
                sc.close();
                return;
            default:
                System.out.println("Invalid choice. Enter 1 to 6.");
        }
    }
}

private static void initializeRooms() {
    rooms.add(new Room(101, "Single", 1000.00));
    rooms.add(new Room(102, "Single", 1000.00));
    rooms.add(new Room(103, "Double", 1800.00));
    rooms.add(new Room(104, "Double", 1800.00));
    rooms.add(new Room(105, "Deluxe", 3000.00));
    rooms.add(new Room(106, "Deluxe", 3000.00));
}

private static void showMenu() {
    System.out.println("\n====================================");
    System.out.println(" HOTEL ROOM RESERVATION AND");
    System.out.println(" GUEST MANAGEMENT SYSTEM");
    System.out.println("====================================");
    System.out.println("1. View All Rooms");
    System.out.println("2. Reserve a Room");
    System.out.println("3. View All Reservations");
    System.out.println("4. Search Reservation / Guest");
    System.out.println("5. Guest Check-out");
    System.out.println("6. Exit");
}

private static void displayRooms() {
    System.out.println("\nROOM LIST");
```

```
        System.out.printf("%-12s %-12s %-13s %-12s%n",
                "Room Number", "Room Type", "Price/Night", "Status");
        for (Room room : rooms) room.displayRoom();
    }

    private static Room findRoom(int number) {
        for (Room room : rooms) {
            if (room.getRoomNumber() == number) return room;
        }
        return null;
    }

    private static Reservation findReservation(int id) {
        for (Reservation r : reservations) {
            if (r.getId() == id) return r;
        }
        return null;
    }

    private static void reserveRoom() {
        displayRooms();
        int number = readInt("Enter room number: ");
        Room room = findRoom(number);

        if (room == null) {
            System.out.println("Room number does not exist.");
            return;
        }
        if (!room.isAvailable()) {
            System.out.println("This room is already occupied.");
            return;
        }

        String name = readNonEmpty("Enter guest name: ");
```

```
        String phone = readPhone();
        String address = readNonEmpty("Enter guest address: ");
        int nights = readPositiveInt("Enter number of nights: ");

        Guest guest = new Guest(name, phone, address);
        Reservation r = new Reservation(nextId, guest, room, nights);
        reservations.add(r);
        room.setAvailable(false);
        nextId++;

        System.out.println("Room reserved successfully.");
        r.display();
    }

    private static void viewReservations() {
        if (reservations.isEmpty()) {
            System.out.println("No active reservations found.");
            return;
        }
        for (Reservation r : reservations) r.display();
    }

    private static void searchReservation() {
        int id = readInt("Enter reservation ID: ");
        Reservation r = findReservation(id);
        if (r == null) {
            System.out.println("Active reservation not found.");
        } else {
            r.display();
        }
    }

    private static void checkOut() {
        int id = readInt("Enter reservation ID for check-out: ");
```

```
        Reservation r = findReservation(id);
        if (r == null) {
            System.out.println("Active reservation not found.");
            return;
        }

        r.display();
        String answer = readNonEmpty("Confirm check-out? (yes/no): ");
        if (answer.equalsIgnoreCase("yes")) {
            r.getRoom().setAvailable(true);
            reservations.remove(r);
            System.out.println("Check-out completed successfully.");
            System.out.println("Room " + r.getRoom().getRoomNumber()
                    + " is now available.");
            System.out.printf("Final Bill: Rs.%.2f%n", r.calculateBill());
        } else {
            System.out.println("Check-out cancelled.");
        }
    }

    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid whole number.");
            }
        }
    }

    private static int readPositiveInt(String prompt) {
        while (true) {
            int value = readInt(prompt);
            if (value > 0) return value;
```

```
            System.out.println("Value must be greater than zero.");
        }
    }

    private static String readNonEmpty(String prompt) {
        while (true) {
            System.out.print(prompt);
            String value = sc.nextLine().trim();
            if (!value.isEmpty()) return value;
            System.out.println("This field cannot be empty.");
        }
    }

    private static String readPhone() {
        while (true) {
            String phone = readNonEmpty("Enter guest phone number: ");
            String digits = phone.replaceAll("\\D", "");
            if (digits.length() >= 7 && digits.length() <= 15
                    && phone.matches("[0-9+() .-]+")) {
                return phone;
            }
            System.out.println("Enter a valid phone number (7-15 digits).");
        }
    }
}
```

6.INPUT AND OUTPUT SCREENS

Sample Run 1 – View Rooms

Input:

1

Output:

ROOM LIST

Room Number Room Type Price/Night Status
101 Single Rs.1000.00 Available

102 Single Rs.1000.00 Available

103 Double Rs.1800.00 Available

Sample Run 2 – Reserve a Room

Input:

2

Room number: 101

Guest name: Kavin

Phone number: 9876543210

Guest address: Coimbatore

Number of nights: 2

Output:

Room reserved successfully.

Reservation ID : 1001

Guest Name : Kavin

Room Number : 101

Total Bill : Rs.2000.00

Sample Run 3 – Search Reservation

Input:

4

Reservation ID: 1001

Output:

Guest Name : Kavin

Phone Number : 9876543210

Room Number : 101

Number of Nights: 2

Total Bill : Rs.2000.00

Sample Run 4 – Guest Check-out

Input:

5

Reservation ID: 1001

Confirm check-out? yes

Output:

Check-out completed successfully.
Room 101 is now available.

Final Bill: Rs.2000.00

Sample Run 5 – Exit

Input:

6
Output:

Thank you for visiting our hotel!

7.DISCUSSION AND CONCLUSION

7.1 Discussion

The Hotel Room Reservation and Guest Management System was developed to simplify room booking and guest record management. It allows hotel staff to view room availability, create reservations, search active bookings, calculate room charges, and check guests out.

The program uses Java classes to represent rooms, guests, and reservations. ArrayList collections store room and active reservation objects during execution, while methods keep the operations organized. Input validation helps reduce errors caused by invalid menu choices or missing information.

Room availability is updated when a reservation is made and restored after check-out. The bill is calculated from the room price and number of nights. The current version is an educational console application and does not save records permanently.

7.2 Conclusion

The Hotel Room Reservation and Guest Management System demonstrates how Core Java can be used to develop a practical hotel management application. It provides basic functions for room reservation, guest details, reservation search, bill calculation, and check-out.

The project meets its main objective of organizing basic hotel operations in a simple menu-driven program. It also demonstrates object-oriented programming, collections, loops, conditional statements, and input validation.

Future improvements may include:

- Database or file-based permanent storage.
- Guest check-in and check-out dates.
- Online booking and payment features.
- User login and management reports.

Overall, the project provides a foundation for a more complete hotel reservation and guest management system.

8.REFERENCES

The following resources were used during the development of this project:

- Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education
- E. Balagurusamy, *Programming with Java*, McGraw-Hill
- Oracle Documentation – Java Programming
- GeeksforGeeks – Java Programming and Data Structures
- TutorialsPoint – Java Tutorial
- W3Schools – Java Tutorial